Duoc UC · Sede Alameda

Ingeniería en Informática

Asignatura PTY4614 · Capstone (Portafolio de Título)

INFORME DE AVANCE DEL PROYECTO APT

AgeCare – Marketplace de Cuidadoras y Fidelización por Puntos en la Consola de Administración

Fase 2 · Desarrollo del proyecto

Integrantes: Bryan Alexander Piña Frías · Julio Lorenzo Muñoz Rojas

Docente: Carlos Andrés Herrera Machuca

Sección: CAPSTONE_002D

Cliente: Wellq Co. · Alloxentric

Fecha: Octubre de 2026

Índice

Abstract (español) 4

Abstract (English) 4

1. Introducción y contexto 5

    1.1 Objetivos 5

2. Metodología de trabajo 6

    2.1 Enfoque 6

    2.2 Prácticas técnicas 6

    2.3 Pertinencia de la metodología 6

3. Desarrollo de ingeniería 7

    3.1 Estado frente al plan de trabajo 7

    3.2 Evidencias de documentación 7

    3.3 Evidencias de programación 7

    3.4 Evidencias de almacenamiento de datos 8

    3.5 Pruebas y estándares de calidad 9

    3.6 Indicadores de calidad por competencia del perfil de egreso 9

4. Ajustes a la propuesta 10

    4.1 Facilitadores y dificultades 10

    4.2 Retroalimentación 10

5. Trabajo en equipo y aporte de cada integrante 10

6. Cómo revisar el proyecto en línea 11

7. Próximos pasos 11

8. Individual conclusions 11

    8.1 Bryan Alexander Piña Frías 11

    8.2 Julio Lorenzo Muñoz Rojas 12

9. Individual reflection 12

9.1 Bryan Alexander Piña Frías 12

9.2 Julio Lorenzo Muñoz Rojas 13

Referencias 14

Anexo A. Evidencias del sistema 15

Abstract (español)

Este informe presenta el avance del Proyecto APT «AgeCare – Marketplace de Cuidadoras y Fidelización por Puntos en la Consola de Administración», desarrollado para Wellq Co. con la contraparte técnica de Alloxentric. El equipo construyó una Consola de Administración (React, FastAPI y PostgreSQL) y un sitio público para familias y cuidadoras, ambos publicados en Vercel con la base de datos y el almacenamiento de archivos en Supabase. Los seis endpoints comprometidos —listado y detalle de cuidadoras, registro de reseñas, suma de puntos, ranking y dashboard de métricas de marketing— están implementados y probados. Además se incorporaron solicitudes de contratación, chat, revisión de documentos por parte del staff y sincronización entre ambos sistemas. Se aplicó una metodología ágil con sprints de dos semanas y despliegue continuo, respaldada por 50 pruebas automáticas. Las principales dificultades fueron de despliegue y de sincronización entre las dos APIs; se resolvieron con ajustes que este informe justifica. Quedan pendientes la demo grabada, la especificación técnica final y la conexión entre el puntaje y la marca «Destacada».

Palabras clave: marketplace, fidelización, API REST, FastAPI, React, PostgreSQL, despliegue continuo.

Abstract (English)

This report presents the progress of the APT Project “AgeCare – Caregiver Marketplace and Points-Based Loyalty in the Administration Console”, developed for Wellq Co. with technical support from Alloxentric. The team built an Administration Console (React, FastAPI and PostgreSQL) and a public website for families and caregivers, both deployed on Vercel with the database and file storage hosted on Supabase. The six committed endpoints (caregiver listing and detail, review registration, points, ranking and a marketing metrics dashboard) are implemented and tested. The team also added hiring requests, chat, document review by the staff and synchronization between the two systems. An agile methodology with two-week sprints and continuous deployment was applied, supported by 50 automated tests. The main difficulties were related to deployment and to keeping the two APIs synchronized; they were solved through adjustments that this report justifies. The recorded demo, the final technical specification and the link between the points and the “Featured” label remain pending.

Keywords: marketplace, loyalty, REST API, FastAPI, React, PostgreSQL, continuous deployment.

1. Introducción y contexto

AgeCare es una plataforma digital de cuidado de adultos mayores desarrollada para Wellq Co. por Xentric Chile SpA (Alloxentric). Dentro de ella, el marketplace de cuidadoras busca resolver un problema concreto: no existe una forma confiable de que una familia compare y elija entre distintas cuidadoras, ni un mecanismo que reconozca y dé visibilidad a quienes prestan un buen servicio. El proyecto se sitúa en Santiago de Chile, en el marco del Capstone realizado con Alloxentric para su cliente Wellq Co., bajo acuerdo de confidencialidad (NDA).

Este informe da cuenta del estado del proyecto en la Fase 2 y se organiza según lo solicitado en la rúbrica de evaluación: ajustes a la propuesta, metodología de trabajo, evidencias de avance y aporte de cada integrante.

1.1 Objetivos

Objetivo general. Construir, dentro de la Consola de Administración de AgeCare, un marketplace de cuidadoras y un sistema de fidelización por puntos y ranking que reconozca el buen desempeño de las cuidadoras y ayude a las familias a elegir con confianza, levantando además la base del frontend de la Consola (React, Vite y TypeScript) conectada al backend ya construido.

Objetivos específicos:

- Levantar la base del frontend de la Consola en React, Vite y TypeScript, conectada al backend FastAPI existente y desplegada en Vercel.
- Exponer un marketplace de cuidadoras visible y ordenable para las familias, reutilizando los endpoints de listado y detalle ya existentes.
- Implementar el endpoint de registro de reseñas para que la familia califique a la cuidadora al finalizar un trabajo.
- Implementar el endpoint que suma puntos automáticamente a la cuidadora por trabajos completados y buenas reseñas.
- Generar un ranking de cuidadoras ordenado por puntaje que priorice su visibilidad.
- Construir un dashboard de métricas de marketing (cuidadoras activas, puntaje promedio y top 5) para el equipo de Wellq Co.

2. Metodología de trabajo

2.1 Enfoque

El equipo trabaja con una metodología ágil basada en Scrum (Schwaber y Sutherland, 2020), organizada en sprints de dos semanas. El backlog se compone de los seis endpoints priorizados en la Fase 1 y cada sprint cierra con un incremento funcional que se revisa antes de seguir. Se extendió el router /admin/marketplace que ya existía en el backend, reutilizando sus roles, su auditoría y su manejo de errores, en lugar de crear uno nuevo.

2.2 Prácticas técnicas

- **Control de versiones:** Git y GitHub, con la rama main como producción y commits descriptivos por cada cambio funcional.
- **Arquitectura por capas:** routers, esquemas de validación (Pydantic), modelos (SQLAlchemy) y servicios de integración separados, siguiendo las recomendaciones de FastAPI (Ramírez, s. f.).
- **Datos versionados:** migraciones con Alembic en la Consola y sincronización idempotente del esquema en el sitio público.
- **Pruebas automáticas:** 50 pruebas con pytest, que se ejecutan sin necesidad de Docker (SQLite) y se repiten en cada cambio.
- **Documentación viva:** documentación interactiva OpenAPI (Swagger) de ambas APIs, README, guía de despliegue y un Registro de cambios con fecha.
- **Despliegue continuo:** cada cambio en main se publica automáticamente en Vercel (Vercel, s. f.), con configuración por variables de entorno y los secretos fuera del código.

2.3 Pertinencia de la metodología

La metodología es pertinente porque el alcance estaba acotado (seis endpoints, dos de ellos ya existentes), porque los sprints cortos permitían validar cada parte antes de continuar y porque trabajar sobre un backend existente obligaba a respetar contratos y reglas ya definidos, como ocurre en una empresa de software. Las pruebas automáticas y el despliegue continuo permitieron detectar y corregir errores en poco tiempo, lo que fue clave cuando el proyecto creció más allá de lo planificado.

3. Desarrollo de ingeniería

3.1 Estado frente al plan de trabajo

Etapa	Semanas	Estado	Resultado
Validación técnica y de alcance	S1–S2	Completado	Backend revisado, entorno de desarrollo montado y alcance fijado en los seis endpoints.
Registro de reseñas	S3–S4	Completado	POST /admin/marketplace/reviews; además, reseña desde el sitio público solo al terminar un trabajo.
Puntos y ranking	S5–S7	Completado	10 puntos por trabajo y 5 más por una reseña de 4 o más estrellas; ranking ordenado por puntaje.
Prioridad en el marketplace	S8–S9	Ajustada	El sitio ordena por destacadas y valoración; falta enlazar el puntaje con la marca «Destacada».
Dashboard de métricas	S10–S11	Completado	GET /admin/marketing/dashboard/summary y vista «Fidelización y ranking».
Cierre e integración	S12–S13	En curso	Despliegue y pruebas listos; faltan demo grabada y especificación técnica final.
Sitio público (actividad agregada)	En paralelo	En curso	Registro, perfiles, documentos, solicitudes, chat y reseñas.

3.2 Evidencias de documentación

- **README.md:** descripción, tecnologías, ejecución local, cuentas de prueba, arquitectura y enlaces para revisar el proyecto en línea.
- **DESPLIEGUE.md:** guía paso a paso de Vercel y Supabase, variables de entorno y solución de problemas.
- **Registro de cambios:** documento (JSON y PDF) con cada cambio del sistema, su fecha y los pasos manuales.
- **Documentación OpenAPI (Swagger):** generada por cada API; muestra los endpoints, sus parámetros y respuestas (Figuras 4 y 5 del anexo).
- **Carpetas Fase 1 y Fase 2:** evidencias académicas, incluido el archivo «Aporte del equipo», generado desde el historial de Git.

3.3 Evidencias de programación

Consola de Administración. La API de la Consola (FastAPI) expone los seis endpoints comprometidos:

Endpoint	Método	Función
/admin/marketplace/caregivers	GET	Lista las cuidadoras del marketplace (ya existía).
/admin/marketplace/caregivers/{id}	GET	Detalle de una cuidadora, con sus documentos (ya existía).

Endpoint	Método	Función
/admin/marketplace/reviews	POST	Registra la reseña de la familia al terminar un trabajo.
/admin/marketplace/caregivers/{id}/points	POST	Suma puntos por trabajo completado o buena reseña.
/admin/marketplace/caregivers/ranking	GET	Ranking de cuidadoras ordenado por puntaje.
/admin/marketing/dashboard/summary	GET	Indicadores: cuidadoras activas, puntaje promedio, top 5.

Sobre esa base se implementaron reglas de negocio validadas en el servidor: una cuidadora solo se publica cuando el staff aprobó su cédula y su certificado de antecedentes; un perfil suspendido se oculta y puede reactivarse; cada acción del staff queda auditada y respeta los permisos por rol. El frontend de la Consola (React 19, Vite y TypeScript) tiene nueve vistas, entre ellas Marketplace de cuidadoras y Fidelización y ranking.

Sitio público. La API del sitio expone 37 endpoints (autenticación con JWT y contraseñas con Argon2, pacientes, perfil de la cuidadora, documentos, marketplace, solicitudes de contratación, chat y reseñas) y su interfaz tiene once páginas. Las reglas más relevantes son: una reseña solo se permite cuando el trabajo terminó; una cuidadora solo puede tener un trabajo activo a la vez; la contratación es una solicitud que la cuidadora acepta o rechaza; los mensajes del chat se eliminan a los 7 días con aviso; y los archivos se limitan a 4 MB en formato PDF, JPG o PNG.

Integración entre sistemas. Ambas APIs se avisan los cambios mediante llamadas servidor a servidor autenticadas con una clave interna (cabecera X-Internal-Key), con reintentos y tiempos de espera. Para que los sistemas no queden con estados distintos si un aviso se pierde, cada lado puede consultar al otro y ponerse al día.

3.4 Evidencias de almacenamiento de datos

La persistencia usa PostgreSQL alojado en Supabase (Supabase, s. f.; The PostgreSQL Global Development Group, s. f.). Entre las dos aplicaciones hay 38 tablas (25 en la Consola y 13 en el sitio público). Las decisiones principales fueron:

- **Modelo de fidelización:** las tablas de reseñas y de movimientos de puntos permiten registrar cada reseña y cada suma de puntos, de modo que el ranking se calcula a partir de datos auditables.
- **Versionado del esquema:** cuatro migraciones de Alembic en la Consola y una sincronización idempotente en el sitio, que agrega las tablas y columnas faltantes al iniciar.

- **Seguridad por filas (RLS):** se activó en las tablas del proyecto para que Supabase no las exponga por su API pública.
- **Archivos:** las fotos y los documentos se guardan en Supabase Storage y la base solo guarda su dirección.
- **Conexiones:** uso del agrupador de conexiones de Supabase, adecuado para funciones sin estado.

Entidad	Para qué se usa
Reseñas y movimientos de puntos (Consola)	Base del ranking y del dashboard de métricas.
Perfil y documentos de la cuidadora	Revisión del staff (pendiente, aprobado o rechazado).
Trabajos activos y solicitudes de contratación	Un trabajo por cuidadora; aceptación o rechazo.
Conversaciones y mensajes	Chat entre familia y cuidadora, con eliminación a los 7 días.

3.5 Pruebas y estándares de calidad

El proyecto cuenta con 50 pruebas automáticas con pytest (30 en la Consola y 20 en el sitio público; Figuras 6 y 7 del anexo) que cubren autenticación, permisos por rol, métricas, soporte, revisión de documentos y los flujos del sitio (solicitudes, chat, reseñas y límite de un trabajo por cuidadora). Se aplican estándares habituales de la industria: API REST documentada con OpenAPI, validación de entradas con Pydantic, errores con formato uniforme y un identificador de petición, autenticación con JWT, contraseñas con Argon2, configuración por variables de entorno y despliegue continuo. Antes de publicar, ambas APIs se prueban en un entorno limpio con la misma versión de Python que usa Vercel.

3.6 Indicadores de calidad por competencia del perfil de egreso

Indicador	Evidencia en el proyecto
1.1 y 1.2 Diseña y aplica pruebas de validación	50 pruebas automáticas con casos de éxito y de error (Figuras 6 y 7).
1.3 Desarrolla mejoras a partir de las pruebas	Los errores hallados en producción se corrigieron y se agregaron pruebas y la verificación en entorno limpio.
2.1 y 2.2 Planifica y controla el proyecto	Plan de trabajo y carta Gantt (Fase 1), monitoreo en la guía 2.4 y ajustes justificados (sección 4).
3.1 y 3.2 Diseña e implementa modelos de datos	38 tablas, migraciones versionadas, RLS y modelo de puntos y reseñas (sección 3.4).
4.1 Construye la solución	Seis endpoints, Consola y sitio público (sección 3.3).
4.2 Integra los componentes	Sincronización entre las dos APIs y conexión de los frontends con sus APIs.
4.3 Implanta la solución	Cuatro proyectos publicados en Vercel con Supabase y despliegue automático.

4. Ajustes a la propuesta

Los ajustes se decidieron a partir de las dificultades y los facilitadores que se presentaron durante el desarrollo. La siguiente tabla indica cada ajuste y su justificación.

Ajuste	Justificación (dificultad o facilitador)
Sitio público ampliado a una pieza completa del sistema	Sin él, las reseñas y los puntos habrían sido datos de prueba sin un origen real; permite probar el flujo completo (registro, aprobación, contratación, reseña, puntos y ranking).
Prioridad en el marketplace: orden por destacadas y valoración	Se priorizó entregar un orden útil para las familias; conectar el puntaje con la marca «Destacada» quedó pendiente por falta de tiempo.
Colección Postman reemplazada por Swagger y 50 pruebas automáticas	Las pruebas automáticas se repiten en cada cambio y verifican casos de éxito y de error; Swagger ya documenta cada endpoint.
Despliegue en Vercel y Supabase con publicación automática	Facilita revisar el proyecto en línea y evita depender de Docker, que se cerraba con frecuencia.
Verificación en entorno limpio antes de publicar	Una API cayó en producción por un error que Python 3.12 no acepta y otra por una dependencia faltante.
Cada API consulta a la otra y se pone al día	Algunos avisos se perdían (instancia dormida o variable sin configurar) y los estados quedaban distintos.
Archivos limitados a 4 MB en Supabase Storage	El plan gratuito de Vercel limita a 4,5 MB el tamaño de una petición.

4.1 Facilitadores y dificultades

Facilitadores: partir de un backend existente, un stack conocido por el equipo, el despliegue automático desde GitHub y las pruebas automáticas.

Dificultades: errores de despliegue (rutas, credencial de la base de datos y una dependencia propia de Windows), diferencias entre el entorno local y producción, avisos perdidos entre las dos APIs y la inestabilidad de Docker Desktop.

4.2 Retroalimentación

La retroalimentación que se reciba del docente a partir de esta evaluación se incorporará en la entrega final del proyecto.

5. Trabajo en equipo y aporte de cada integrante

El trabajo se repartió por áreas para evitar conflictos en el código, y todo el desarrollo está en el mismo repositorio de GitHub, donde se identifica el aporte de cada integrante. El detalle commit por commit está en el archivo «Fase 2/Aporte del equipo.md», que se genera automáticamente desde el historial de Git.

Integrante	Áreas de trabajo
Bryan Piña Frías	Consola de Administración y su API, API del sitio público, sincronización entre ambas, chat, solicitudes de contratación, revisión de documentos, pruebas automáticas y despliegue en Vercel y Supabase.
Julio Muñoz Rojas	Sitio público: perfil profesional de la cuidadora, foto de perfil, documentos, página de inicio, insignia de cuidadora verificada y ficha de adultos mayores con cuidadora.

6. Cómo revisar el proyecto en línea

Recurso	Enlace
Sitio público (familias y cuidadoras)	https://bryan-pi-a-julio-munoz-proyecto-de-sigma.vercel.app/
Consola de Administración	https://bryan-pi-a-julio-munoz-proyecto-de-omega.vercel.app/login
Documentación de la API del sitio	https://agecare-api-general.vercel.app/api/v1/docs
Documentación de la API de la Consola	https://bryan-pi-a-julio-munoz-proyecto-de.vercel.app/api/v1/admin/docs
Repositorio	https://github.com/Bemol69/BryanPi-a_JulioMunoz_ProyectoDeTitulo_AgecARE

Las cuentas de prueba (familia, cuidadora y staff de la Consola) están en el README del repositorio y se muestran también en las páginas de inicio de sesión.

7. Próximos pasos

- Grabar la demo del flujo completo: una reseña suma puntos y la cuidadora sube en el ranking.
- Redactar la especificación técnica final de los endpoints en un documento propio.
- Conectar el puntaje de la Consola con la marca «Destacada» del sitio público.
- Congelar funcionalidades nuevas y mantener las pruebas automáticas al día hasta el cierre.

8. Individual conclusions

8.1 Bryan Alexander Piña Frías

AgeCare has moved from a plan to a working system. The six endpoints we committed to in Phase 1 are implemented and tested, and the Administration Console is live on Vercel with its data in Supabase. The most valuable technical lesson for me was data modelling: deciding how reviews, points and documents relate to each other and enforcing the business rules on the server. A review is only allowed after a job has ended, a caregiver is published only after the staff approves her documents, and a caregiver can only have one active job. Putting those rules in the backend and covering them with automated tests made the system safer to change.

The second lesson is that an application that works on my computer is not yet a finished product. Most of my difficulties appeared during deployment: routing errors, a Python version that rejected code my local version accepted, a missing dependency and notifications that were lost between the two APIs. Solving them taught me to test in a clean environment before publishing and to design the integration so that each system can recover on its own.

The pending items are the recorded demo, the final technical specification and the link between the points ranking and the “Featured” label. Until the final delivery, I also want to keep working in small and frequent commits so that the contribution of each team member stays visible in the repository.

8.2 Julio Lorenzo Muñoz Rojas

In this phase my work focused on the public website, the part of AgeCare that families and caregivers actually use. I built the caregiver's professional profile form, the profile photo upload, the document upload with its review status and the redesign of the home page. More recently I added the verified-caregiver badge, the profile completeness indicator and the marker that shows which older adults already have a caregiver in the family view. Each of these features touches the database, the API and the interface, so I worked across the whole stack with FastAPI and PostgreSQL, as I had planned in Phase 1.

The result can be seen in the system: a caregiver can complete her profile, upload her documents and check whether the staff approved them, and the staff can review them from the Console. Some of my changes to the profile form also affected the synchronization between the two backends, so I had to keep both sides consistent. This showed me that the data a user uploads must be validated and traceable: each document has a status and the system keeps the approved file until the staff accepts a new one.

For the final delivery I want to review the complete caregiver flow from registration to hiring requests and support the recorded demo, so that the work of the whole team is easy to follow.

9. Individual reflection

9.1 Bryan Alexander Piña Frías

At the beginning of the semester I described my strengths as databases, backend development and project coordination, and my main weakness as writing complete applications from scratch. This phase tested both. Building the console, two APIs and a public website forced me to write much more application code than I expected, and I now feel more confident doing it. I also learned that planning is

not only a Gantt chart: the scope grew when we added the public website, and I had to adjust the plan instead of ignoring the change.

If I could start again, I would document while I build, because I left the technical specification and the demo for the end, and I would test in an environment similar to production before every release. What worked well was working in small steps, automated tests and continuous deployment, because they let me see problems early.

Working in a pair requires clear agreements. We split the work by areas, which avoided conflicts in the code, but we can improve the code review between us and keep a steadier pace until the final delivery. From this phase I take a stronger interest in the deployment side of software engineering, which I had not considered before, together with my original interest in backend development and databases.

9.2 Julio Lorenzo Muñoz Rojas

In Phase 1 I said that my strengths were data modelling, SQL and Python, and that I needed to improve testing and quality, security and project management. This phase gave me practice in some of these areas. Designing the document upload showed me that data quality starts when the data enters the system: I had to decide which file types are accepted, which size is allowed and how to show the review status to the user. This connects with my interest in data governance, because it is about traceability and trustworthy information.

I also understood why automated tests matter. The team now has 50 of them, and they allow us to change rules without breaking previous features. Until now I had mostly tested manually, so in the next weeks I want to learn to write tests myself, starting with the profile and document flows I worked on.

The main difficulty for me was time: combining the project with work, other courses and the search for a professional internship. We handled it by dividing the tasks week by week, and when one of us had a heavier week the other moved some tasks forward. For the final stretch I plan to leave a margin before each delivery for tests and corrections, and to document better what I do.

Regarding teamwork, we split the work by areas: I took the caregiver's profile and the home page, and Bryan took the console, the APIs and the deployment. Communication has been good and we ask each other for help when something does not work, but we could hold follow-up meetings more regularly and keep a record of what we decide. From this phase I keep the idea that I like building features that handle real user data, which is close to my interest in data work.

Referencias

PostgreSQL Global Development Group, The. (s. f.). PostgreSQL documentation. Recuperado el 19 de octubre de 2026, de https://www.postgresql.org/docs/

Ramírez, S. (s. f.). FastAPI. Recuperado el 19 de octubre de 2026, de https://fastapi.tiangolo.com/

Schwaber, K. y Sutherland, J. (2020). La guía de Scrum. Recuperado el 19 de octubre de 2026, de https://scrumguides.org/

Supabase. (s. f.). Supabase documentation. Recuperado el 19 de octubre de 2026, de https://supabase.com/docs

Vercel. (s. f.). Vercel documentation. Recuperado el 19 de octubre de 2026, de https://vercel.com/docs

Anexo A. Evidencias del sistema

Las figuras son capturas de pantalla del sistema en ejecución con datos de prueba.

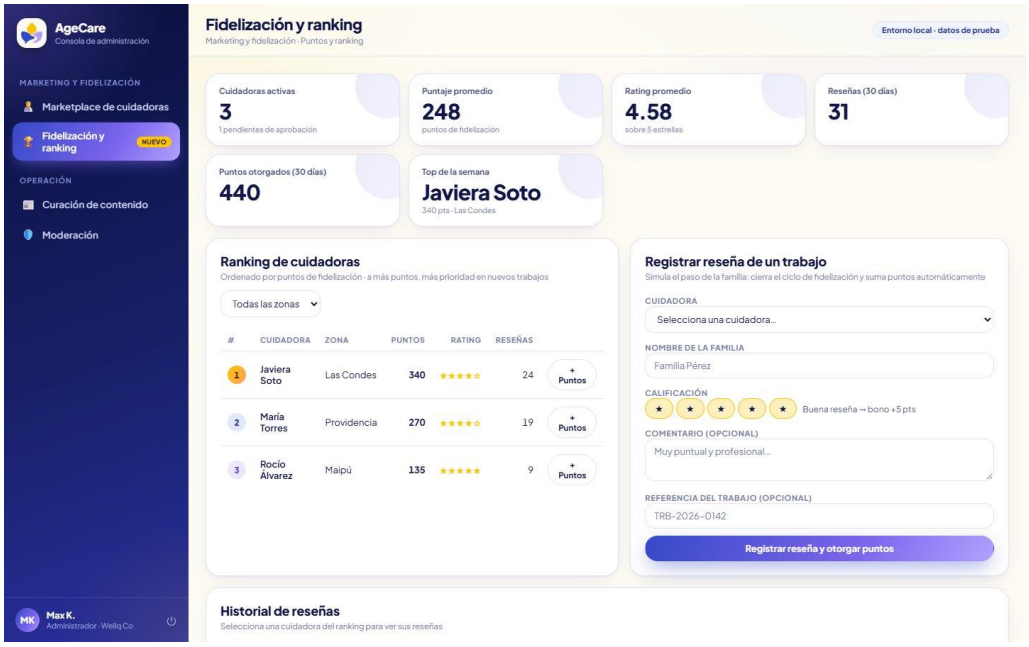


Figura 1. Consola de Administración: Fidelización y ranking (indicadores del dashboard, ranking y registro de reseñas).

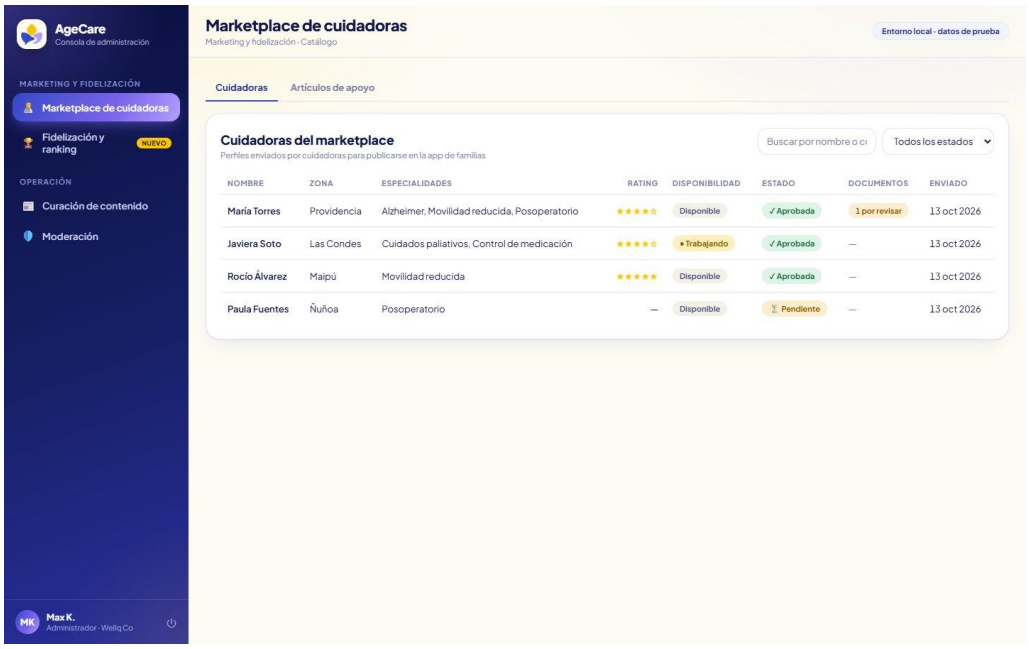


Figura 2. Consola de Administración: marketplace de cuidadoras con estado de aprobación y documentos.

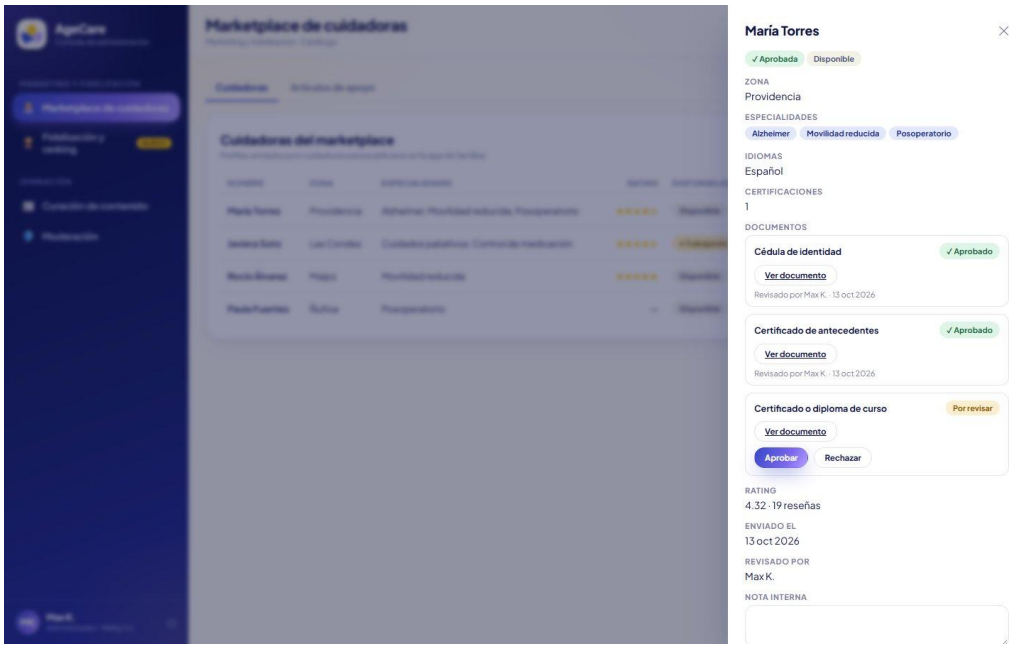


Figura 3. Consola de Administración: ficha de una cuidadora y revisión de documentos.

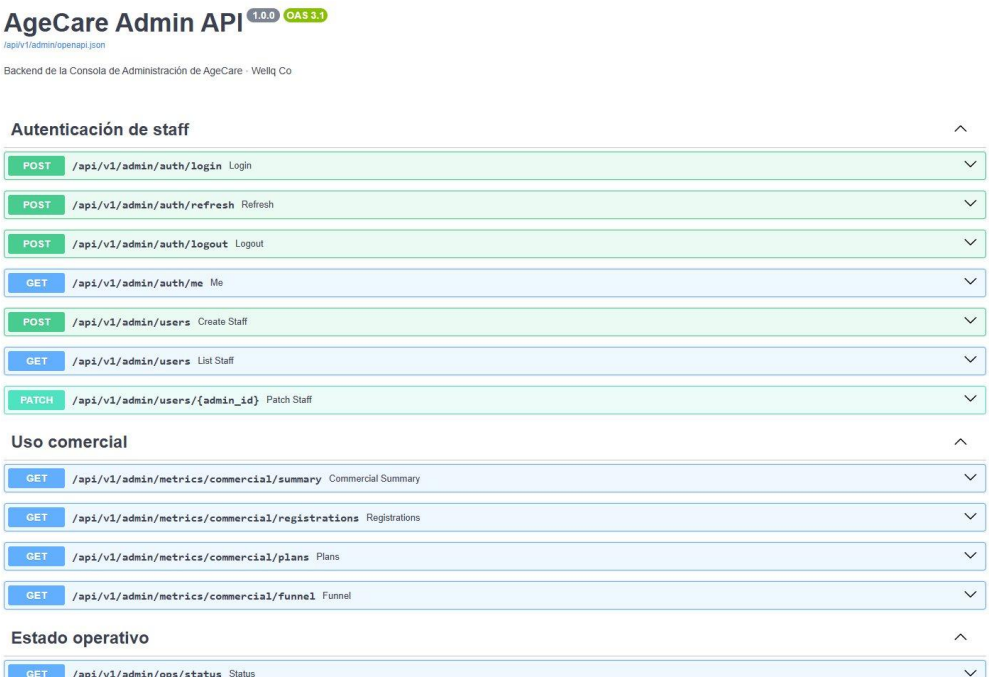


Figura 4. Documentación interactiva (Swagger) de la API de la Consola.

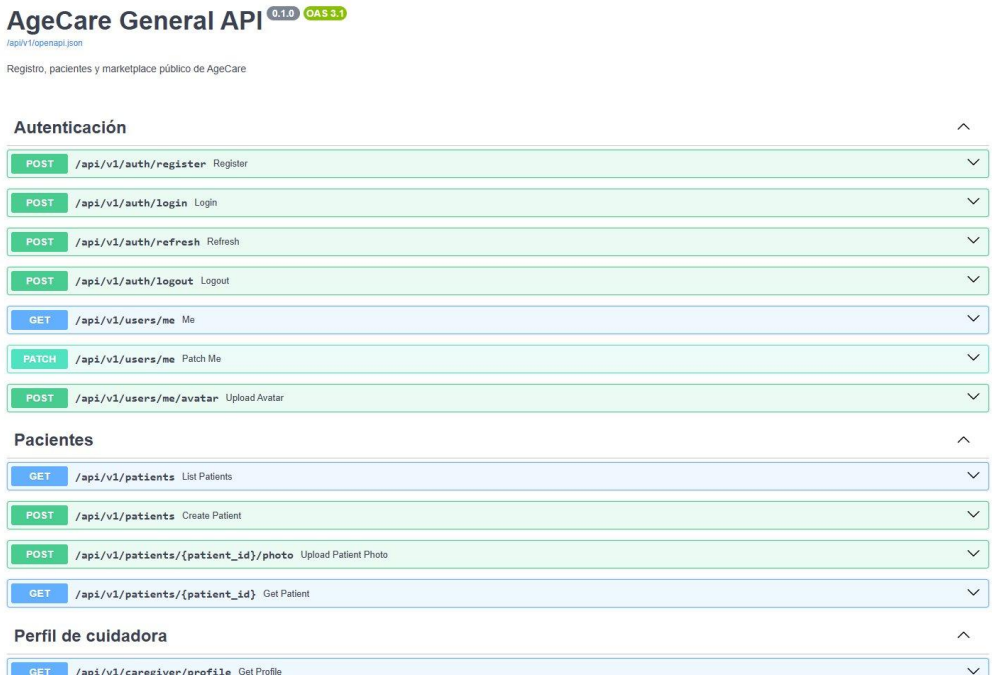


Figura 5. Documentación interactiva (Swagger) de la API del sitio público.

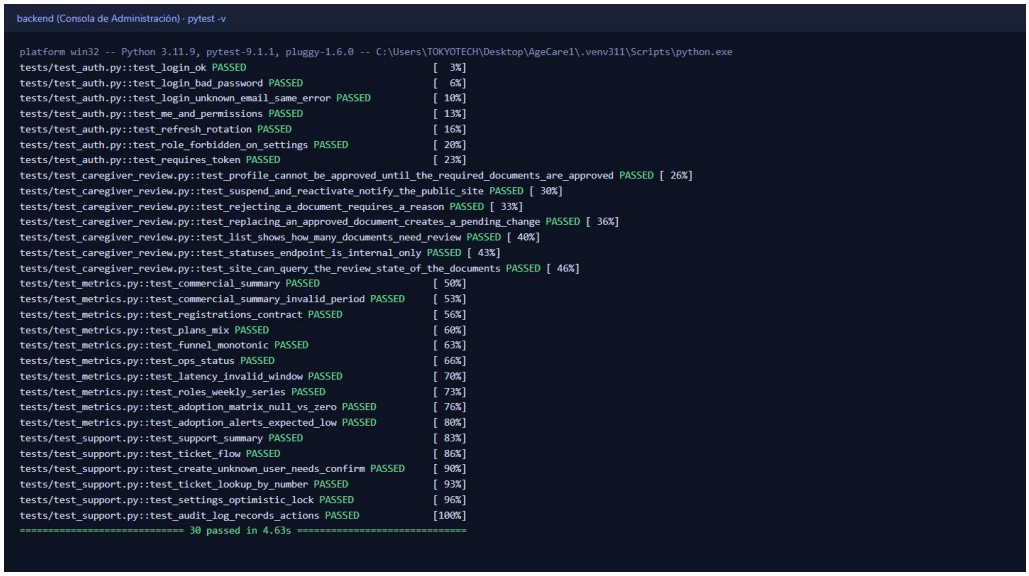


Figura 6. Pruebas automáticas de la Consola: 30 pruebas aprobadas.

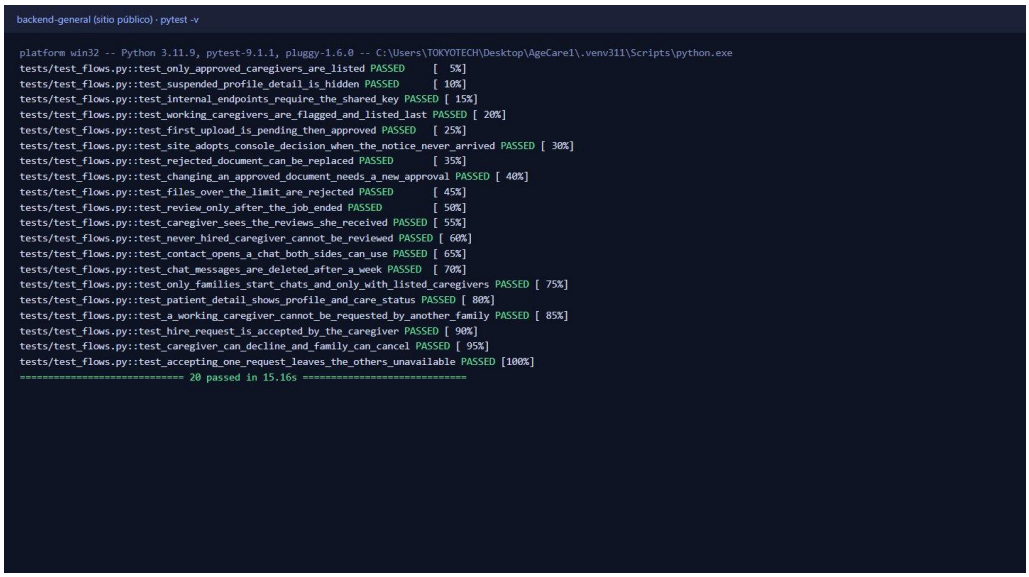


Figura 7. Pruebas automáticas del sitio público: 20 pruebas aprobadas.

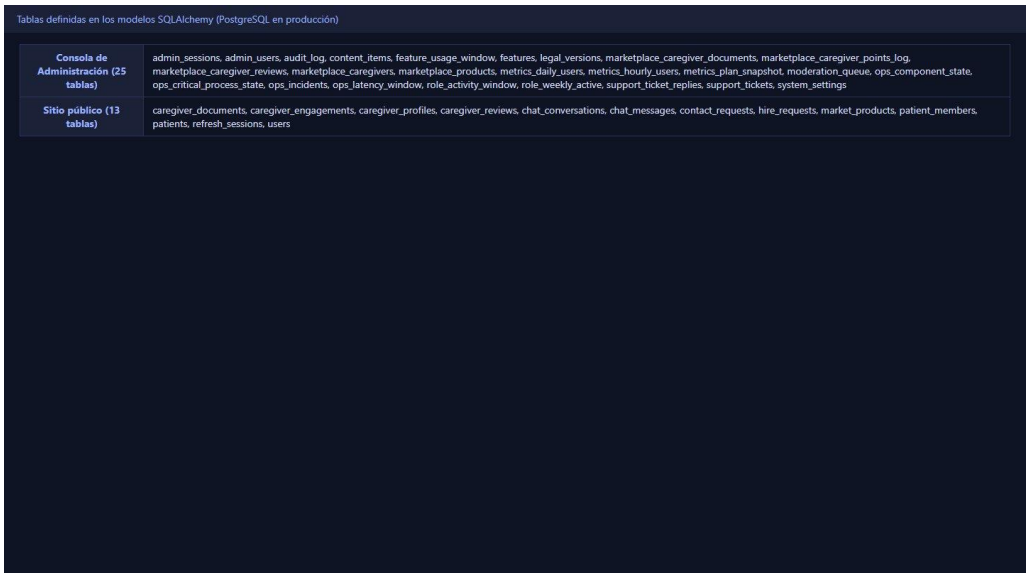


Figura 8. Tablas del modelo de datos de las dos aplicaciones (38 en total).

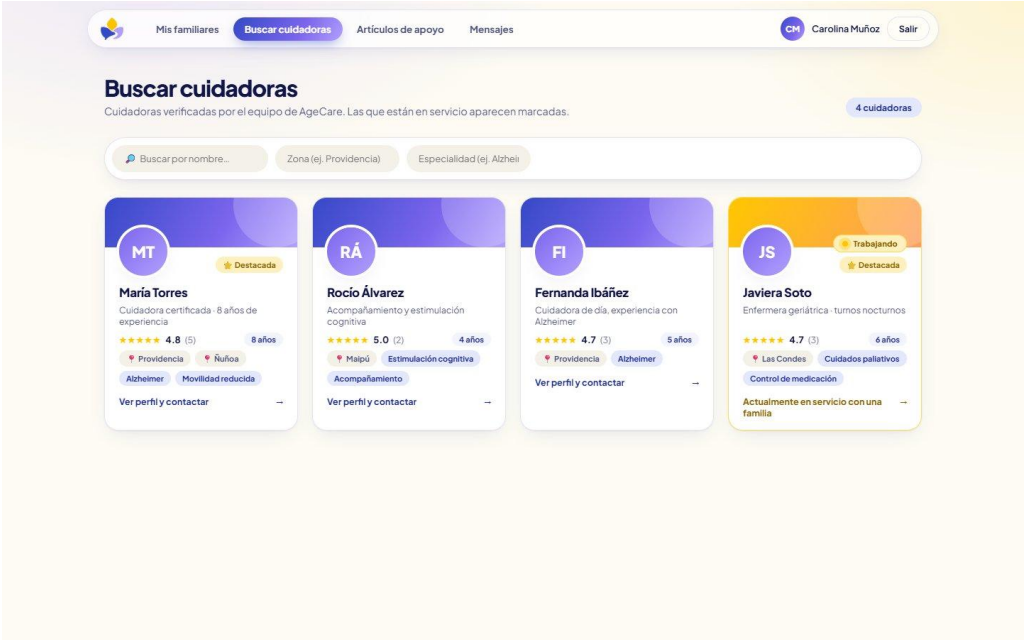


Figura 9. Sitio público: búsqueda de cuidadoras (la marca «Trabajando» indica quién tiene un trabajo activo).

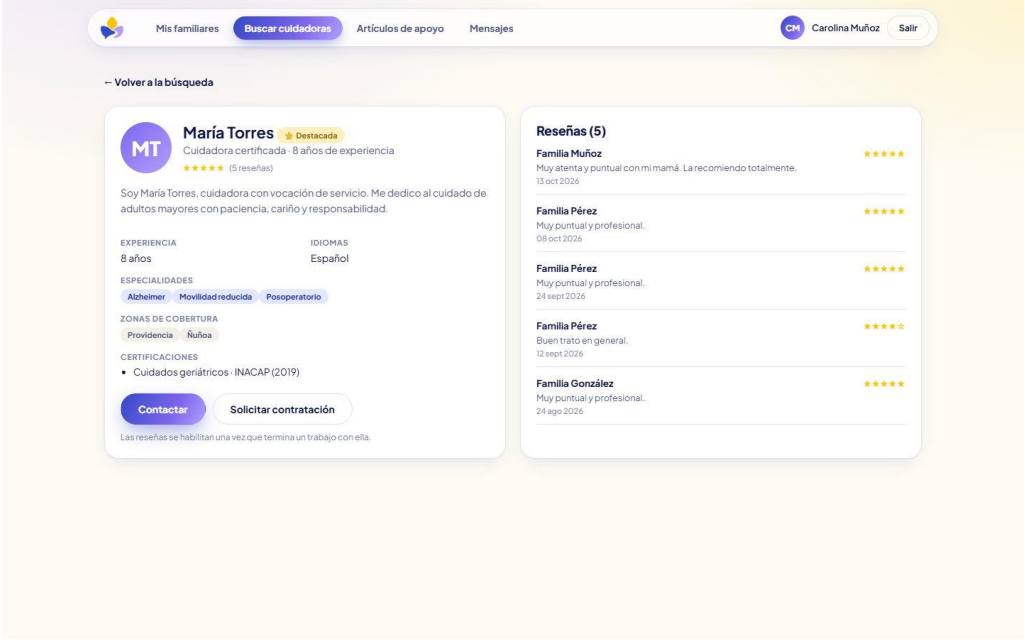


Figura 10. Sitio público: perfil de la cuidadora con sus reseñas.

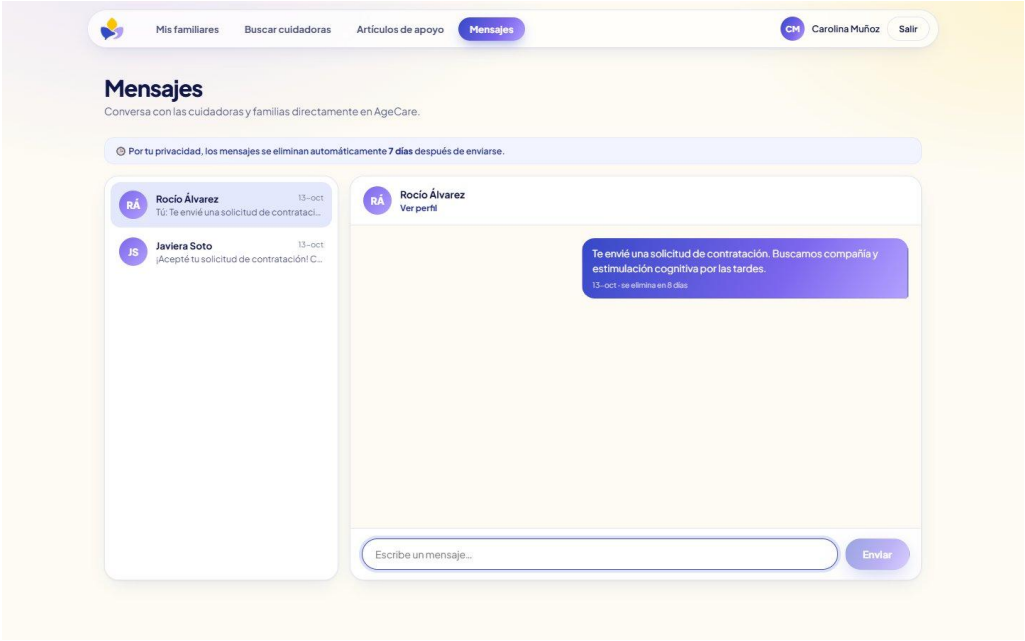


Figura 11. Sitio público: chat entre familia y cuidadora.

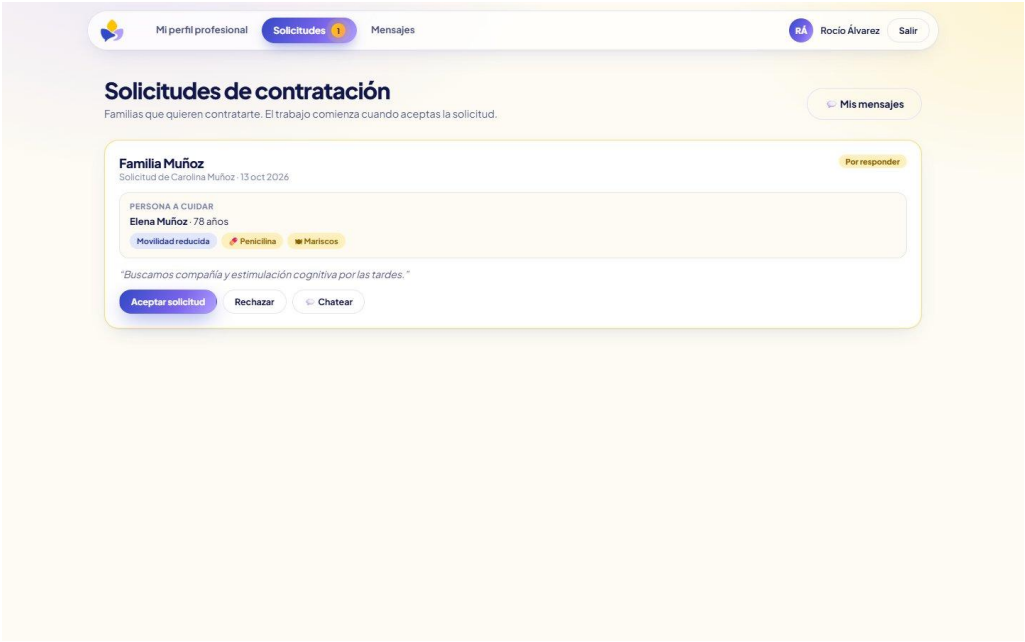


Figura 12. Sitio público: solicitud de contratación recibida por la cuidadora.

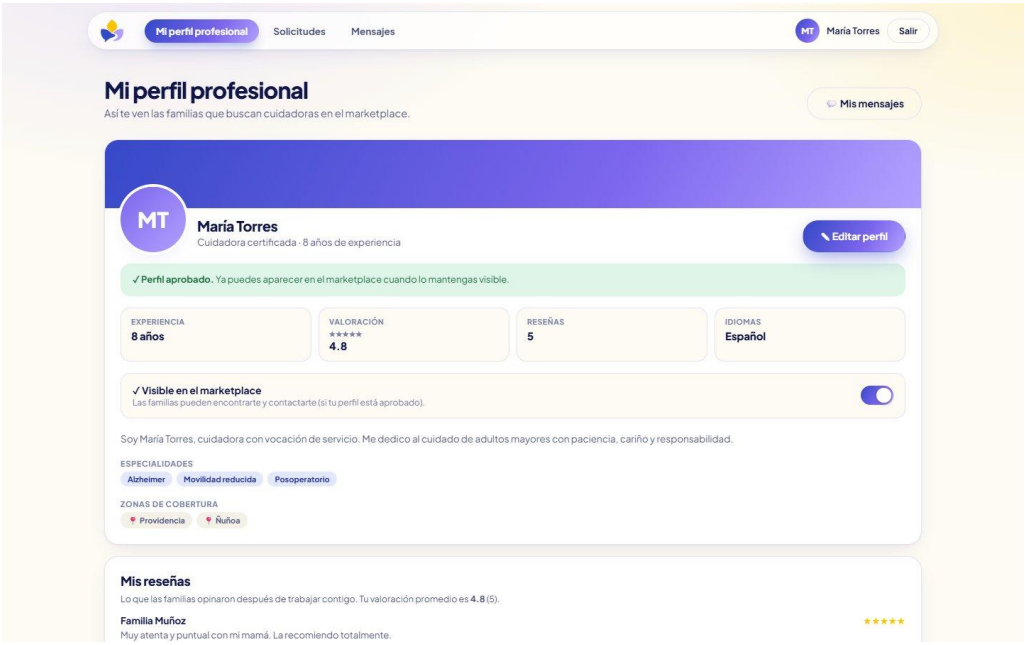


Figura 13. Sitio público: perfil profesional de la cuidadora.

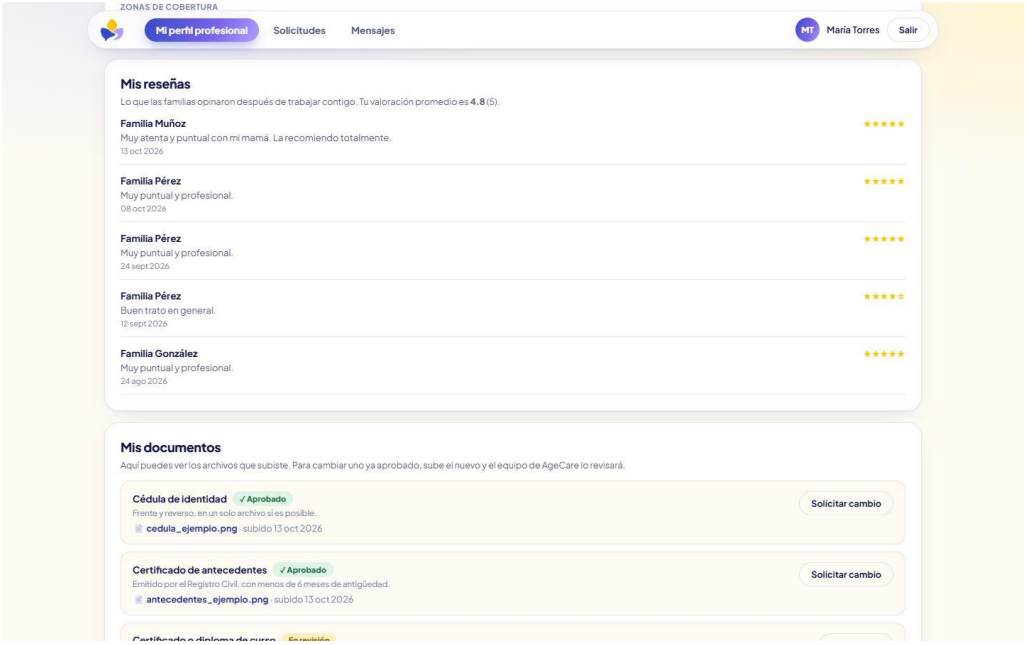


Figura 14. Sitio público: documentos de la cuidadora con su estado de revisión.

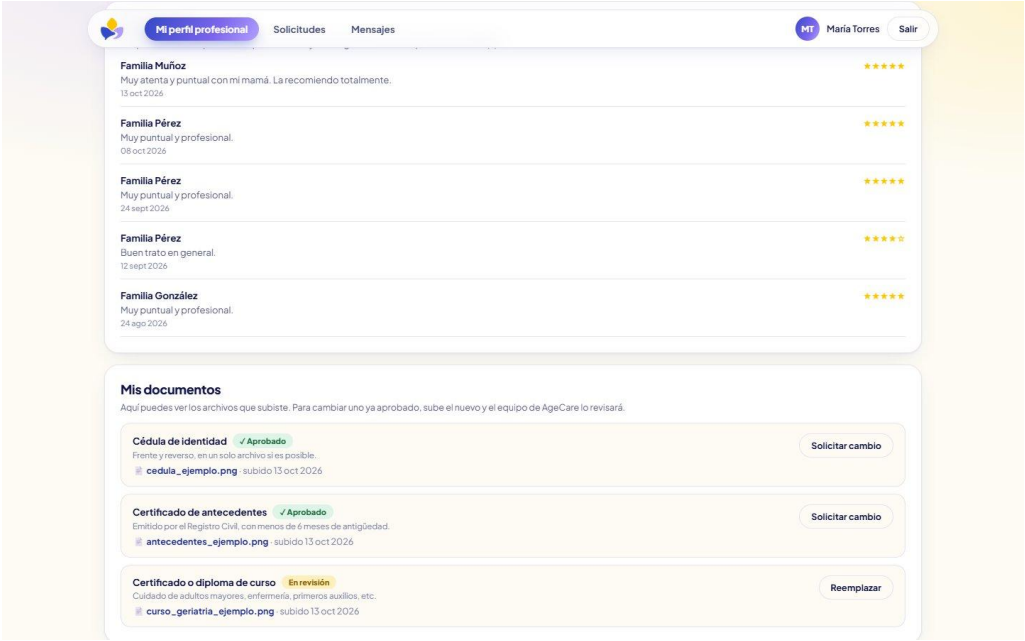


Figura 15. Sitio público: la cuidadora ve las reseñas que le han dejado.

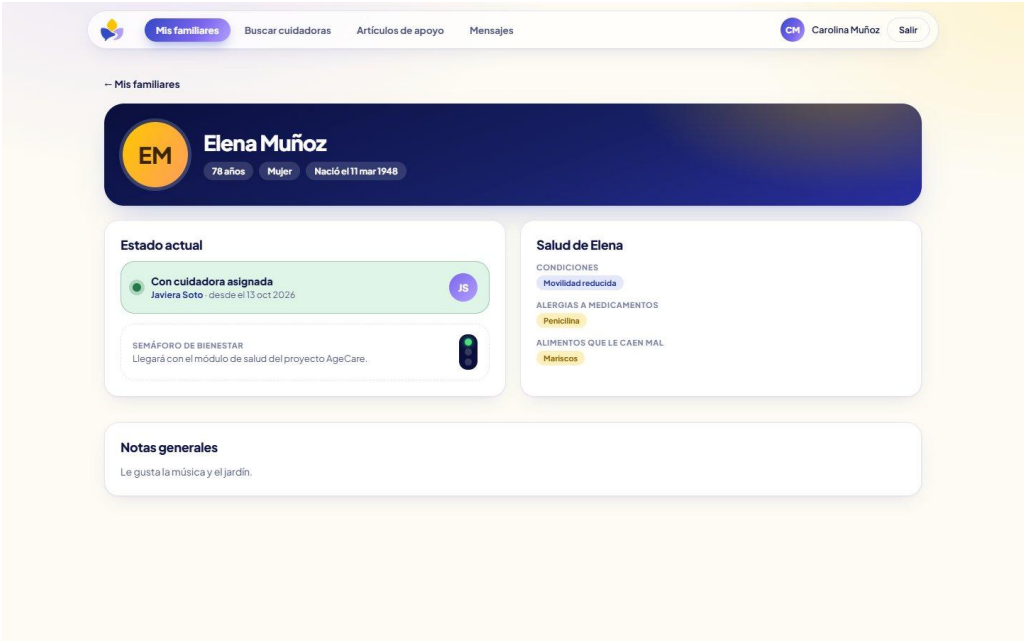


Figura 16. Sitio público: ficha del familiar con su estado de cuidado.

```
git log --format='%h %ad %an %s' (rama main)

66211ad  2026-10-12  Bryan Alexander Piña Frias  La cuidadora puede ver las reseñas que le han dejado
e9c8b4e  2026-10-12  Bryan Alexander Piña Frias  El sitio toma de la Consola las decisiones sobre documentos que no alcanzaron a llegar
4f569b7  2026-10-12  Bryan Alexander Piña Frias  Arregla el arranque de las APIs en Vercel (orden de clase en esquema y httpx en API Admin)
42cd902  2026-10-12  Bryan Alexander Piña Frias  Solicitud de contratación que la cuidadora acepta, con el chat conectado
ddf27a3  2026-10-12  Bryan Alexander Piña Frias  Una cuidadora solo puede tener un trabajo activo a la vez
7891945  2026-10-12  Bryan Alexander Piña Frias  Mejoras de la lista "Mejoras AgeCare": ficha del familiar, chat, revisión de documentos y perfiles
058a9a8  2026-10-06  Bryan Alexander Piña Frias  Dependencia de Windows de rolldown pasa a optionalDependencies para que instale en Linux (Vercel)
b56e06c  2026-10-06  Bryan Alexander Piña Frias  Vercel: usar routes (conserva la ruta original) en vez de rewrites para las APIs
e4ee92d  2026-10-02  Bryan Alexander Piña Frias  RLS solo sobre las tablas de AgeCare y aviso al reutilizar un proyecto de Supabase
07ecf51  2026-10-02  Bryan Alexander Piña Frias  Script para crear tablas en Supabase sin exponer la cadena de conexión
0bd94a0  2026-10-02  Bryan Alexander Piña Frias  Preparar despliegue en Vercel + Supabase
06341b5  2026-10-02  Bryan Alexander Piña Frias  Corrige espacio bajo los títulos de tarjeta en el panel de cuidadora
1d6cf4a  2026-10-02  Bryan Alexander Piña Frias  Rediseño visual con paleta del logo y preparación para Vercel
6e4a4db  2026-09-26  JulioMunozz  Agregar diario de reflexión Fase 1 de Julio
6576c4b  2026-09-26  JulioMunozz  Agregar autoevaluación de competencias de Julio (Fase 1)
b25cf91  2026-09-26  Bryan Alexander Piña Frias  Completar diario de reflexión Fase 1 de Bryan
d1eda77  2026-09-26  Bryan Alexander Piña Frias  Agregar autoevaluación de competencias de Bryan (Fase 1)
642fa55  2026-09-26  Bryan Alexander Piña Frias  Reorganizar evidencias de Fase 1 y completar README según formato solicitado por Duoc UC
fa16547  2026-09-22  JulioMunozz  Mejora panel cuidadoras e integración de documentos
d8e6218  2026-09-22  Bryan Alexander Piña Frias  Nuevos Cambios
7812e9d  2026-09-16  JulioMunozz  mejora en formulario de mi perfil profesional
0d4bd98  2026-09-15  JulioMunozz  Cambios en la homepage
5c59097  2026-09-15  JulioMunozz  Agregando foto de perfil
d06f581  2026-09-14  Bryan Alexander Piña Frias  Agregar Consola de Administracion y sitio publico de AgeCare
c073ab7  2026-09-06  Bryan Alexander Piña Frias  Add files via upload
93384ab  2026-09-06  Bryan Alexander Piña Frias  Initial commit
```

Figura 17. Historial de commits del repositorio.

AgeCare - Registro de cambios

Historial de todo lo que se ha cambiado en el proyecto (sitio de familias y cuidadoras, Consola de administración y APIs). Lo más reciente está primero.

2026-10-12

La cuidadora ve las reseñas que le han dejado

En su perfil profesional hay una sección "Mis reseñas" con la opinión de cada familia.

Sitio de cuidadoras

- Nueva tarjeta "Mis reseñas" en su perfil: promedio, cantidad y cada reseña con el nombre de la familia, las estrellas, el comentario y la fecha (la más nueva primero).
- El contador "Reseñas" de su perfil lleva directo a esa sección. Si aún no tiene, se explica cuándo aparecerán.

Backend y base de datos

- Nuevo endpoint GET /caregiver/reviews, que entrega solo las reseñas de la cuidadora que lo consulta. Prueba automática nueva (sitio: 20).

2026-10-12

Los documentos aprobados en la Consola ahora se reflejan en el sitio de la cuidadora

Si el aviso de la Consola no llegaba al sitio, la cuidadora seguía viendo sus documentos "En revisión" aunque ya estuvieran aprobados. Ahora el sitio consulta la decisión del staff y se actualiza solo.

Sitio de cuidadoras

- Al abrir "Mis documentos", el sitio revisa en la Consola si el staff ya aprobó o rechazó cada archivo y actualiza el estado (y el motivo, si fue rechazado).
- Solo adopta la decisión si se refiere al mismo archivo que la cuidadora tiene en el sitio; un archivo nuevo subido después sigue en revisión.

Backend y base de datos

- Nuevo endpoint interno en la Consola para consultar el estado de los documentos de una cuidadora (solo servidor a servidor, con la clave interna).
- Pruebas automáticas nuevas (sitio: 19, Consola: 30).

2026-10-12

Corrección: error al iniciar sesión ("Failed to fetch") tras la última actualización

Las dos APIs en Vercel dejaron de arrancar con el último despliegue y el login del sitio y de la Consola mostraba "Failed to fetch". Quedó corregido.

Backend y base de datos

- API General: una clase del esquema se usaba antes de definirse; Python 3.12 (el de Vercel) lo rechaza al arrancar. Se reordenó.

Figura 18. Registro de cambios del proyecto.

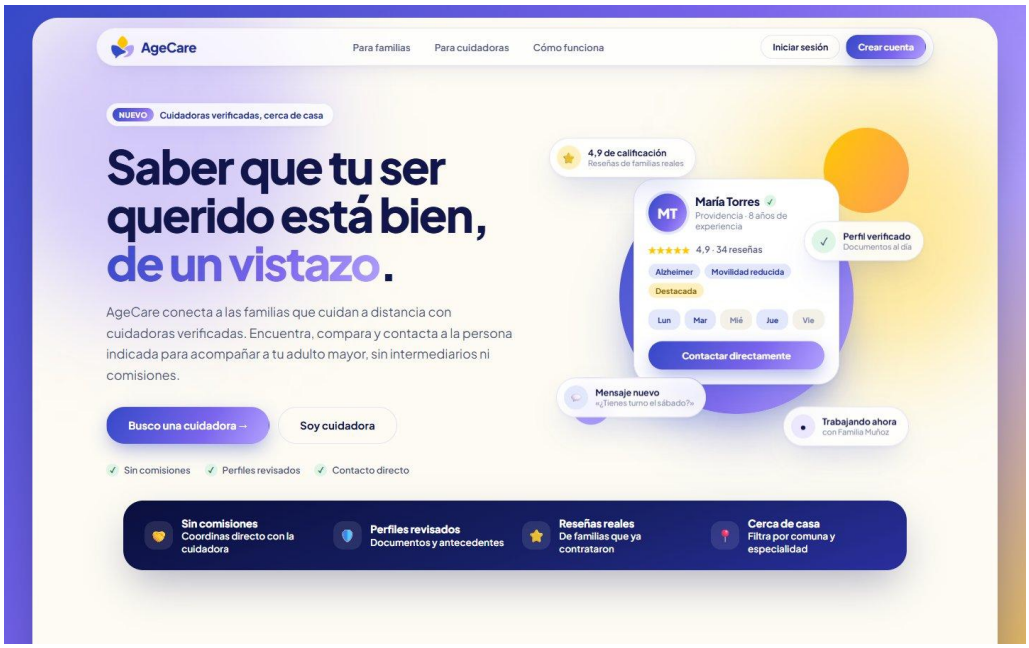


Figura 19. Sitio público publicado en Vercel.

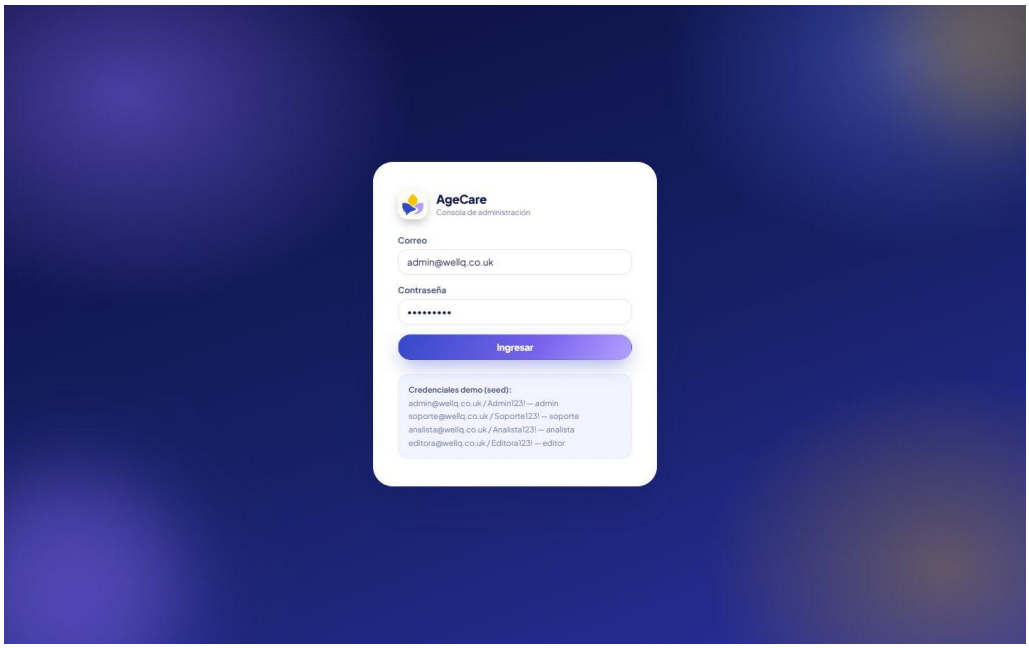


Figura 20. Consola de Administración publicada en Vercel.